SIMATS ENGINEERING

ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO4**	**Assessment:**	**ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK**
Weightage:	**30%**	**Total Marks:**	**25**
CO4 Statement:	*Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.*		
Student Name:	**Deepa preya H**	**Reg.No.:**	**192421047**
Department:	**Information technology**	**Date:**	**20-08-2026**

ANNEXURE A
INDUSTRY PROBLEM SOLVING TASK

1. Smartphone Performance Optimization
   A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

2. Cloud Server Memory Bottleneck
   A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

3. Autonomous Vehicle Data Processing
   An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

4. AI Inference Edge Device
   An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

5. High-Performance Gaming Console
   A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

***Code of Conduct**:*

*I **Deepa preya H / 192421047** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

1. Smartphone Performance Optimization

Problem Statement

A mobile manufacturer observes lag while switching between apps. The main reason can be inefficient memory access and delays in fetching instructions and data from different levels of the memory hierarchy. The roles of L1, L2, L3 cache and DRAM must therefore be analyzed, and a suitable hierarchical memory redesign should be proposed to reduce latency and improve throughput.

1. Role of Different Memory Levels

Memory	Location/Role	Latency	Capacity	Main Advantage
L1 Cache	Closest to CPU core	Very low	Very small	Fastest access
L2 Cache	Near/associated with CPU core	Low	Larger than L1	Reduces L1 misses
L3 Cache	Shared among CPU cores	Higher than L1/L2	Larger	Reduces DRAM accesses
DRAM	Main memory	High compared with cache	Large	Stores active applications and data

2. How the Lag Occurs

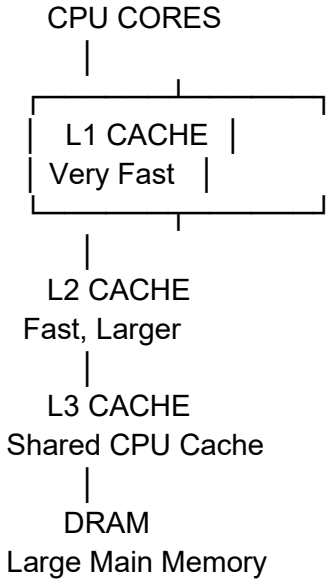
When a user switches from one application to another:

CPU → L1 Cache → L2 Cache → L3 Cache → DRAM

- If the required data is found in **L1**, access is extremely fast.
- If there is an **L1 miss**, the processor checks L2.
- If L2 also misses, it checks L3.
- If all cache levels miss, data must be fetched from DRAM, which has much higher latency.
- Frequent DRAM accesses during application switching can therefore cause noticeable delays.

3. Proposed Hierarchical Memory Redesign

The manufacturer should use the following hierarchy:



4. Improvements Proposed

a) Increase L1 cache efficiency

A sufficiently sized L1 instruction and data cache should be used so that frequently accessed instructions and data remain close to the CPU.

b) Optimize L2 cache

A larger and efficient L2 cache can store more application data and reduce the number of accesses to L3 and DRAM.

c) Use an adequately sized shared L3 cache

L3 cache can retain commonly used data when switching between applications. This is particularly useful in a multi-core smartphone processor.

d) Improve DRAM

Use high-bandwidth, low-power mobile DRAM so that when data is not available in cache, it can be transferred quickly while maintaining smartphone power efficiency.

e) Improve cache management

The processor can use effective cache replacement and prefetching techniques to predict frequently required data and load it into cache before the CPU requests it.

5. Expected Performance Improvement

The redesigned hierarchy reduces the frequency of expensive DRAM accesses.

Without optimization:

CPU → L1 Miss → L2 Miss → L3 Miss → DRAM

↓

High access latency

↓

App-switching lag

With optimization:

CPU → L1/L2/L3 Cache → Required Data

↓

Low access latency

↓

Faster app switching

This improves:

- **Latency:** More data is obtained from faster cache levels.
- **Throughput:** The CPU spends less time waiting for memory.
- **Application switching:** Frequently used application data can remain available in cache/DRAM.
- **Power efficiency:** Reducing unnecessary DRAM accesses can also reduce memory-system activity.

6. Final Recommended Design

The recommended smartphone memory hierarchy is:

CPU → Optimized L1 → Larger/Faster L2 → Shared L3 → High-bandwidth DRAM

The design should focus on keeping **frequently accessed data in L1/L2**, using **L3 as a larger shared cache**, and using **high-bandwidth DRAM** for less frequently accessed application data. This hierarchical approach reduces memory-access latency and improves overall system throughput.

Conclusion
The lag during app switching is mainly associated with delays when required data is not available in the faster cache levels and must be obtained from DRAM. A properly designed **L1–L2–L3 cache hierarchy combined with efficient DRAM** can significantly reduce these delays. Therefore, optimizing cache sizes, cache management, prefetching and DRAM bandwidth provides a practical solution for improving smartphone performance. This directly addresses the assessment requirement to differentiate memory levels based on **latency, bandwidth and throughput**.

2. Cloud Server Memory Bottleneck
Problem Statement
A cloud service provider experiences high latency in database queries. The problem requires comparing cache hierarchy with virtual memory paging and recommending suitable memory technologies to improve performance.

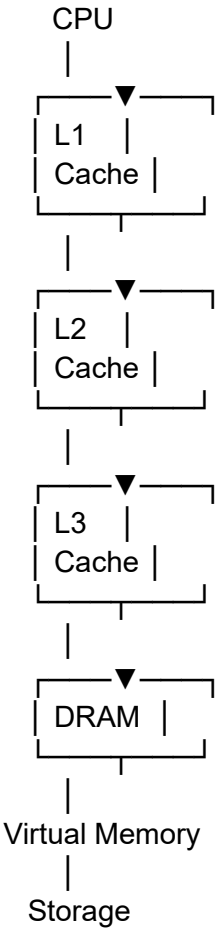
1. Cause of the Problem
Database queries frequently access data and instructions. If the required data is not available in the faster cache levels, the processor has to access lower levels of memory.
A simplified path is:
CPU
↓
L1 Cache
↓
L2 Cache
↓
L3 Cache
↓
DRAM
↓
Virtual Memory / Storage
When the system frequently accesses virtual memory, **page faults** can occur. The operating system then has to transfer pages between DRAM and storage, increasing query latency.

2. Cache Hierarchy vs Virtual Memory Paging

Feature	Cache Hierarchy	Virtual Memory Paging
Main purpose	Reduce CPU memory-access latency	Extend available memory using storage
Managed by	Hardware/cache controller	Operating system + hardware
Levels	L1, L2, L3	Pages in virtual and physical memory
Speed	Very fast	Much slower when storage is accessed
Unit of transfer	Cache line	Memory page
Main benefit	Faster repeated data access	Allows programs to use more memory than physical DRAM
Problem when inefficient	Cache misses	Page faults
Effect on database	Faster frequently accessed queries	Frequent paging can cause very high latency

3. Cache Hierarchy

The cloud server should use a multi-level cache:



Frequently accessed database information can benefit from being available closer to the CPU, reducing the number of slower DRAM accesses.

4. Virtual Memory Paging

Virtual memory allows applications to use an address space larger than the available physical memory.

When required data is already present in DRAM:

CPU → DRAM → Data

But when the required page is not present:

CPU → Page Fault → Storage → DRAM → CPU

This additional transfer creates significant latency.

Therefore, for a database server with heavy memory requirements, **frequent paging should be minimized**.

5. Recommended Memory Technologies

The cloud provider should use:

High-performance cache + high-capacity DRAM + fast storage

L1/L2/L3 Cache

Use fast **SRAM-based cache** for frequently accessed instructions and database-related data.

DRAM

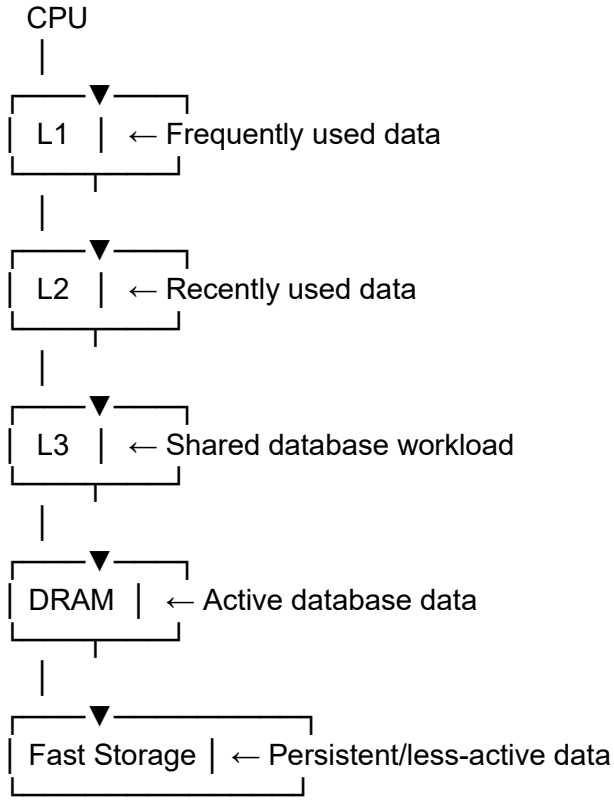
Use sufficiently large, high-bandwidth **DRAM** as the primary working memory for databases.

Storage

Use fast solid-state storage for persistent data and virtual-memory backing storage. However, storage should not be treated as a replacement for adequate DRAM because storage access is significantly slower.

6. Proposed Solution

The recommended architecture is:



7. Improvements

1. Increase cache capacity to keep frequently accessed data closer to the CPU.
2. Increase DRAM capacity to reduce dependence on virtual-memory paging.
3. Use high-bandwidth DRAM to improve data transfer rates.
4. Reduce page faults by keeping actively used database pages in physical memory.
5. Use fast storage for persistent data and as virtual-memory backing storage.
6. Optimize memory allocation so that frequently accessed database data remains in faster memory.

8. Expected Result

The proposed architecture reduces:

- Cache miss penalties
- DRAM access frequency
- Page faults
- Storage accesses
- Database query latency

At the same time, it improves:

- Memory bandwidth
- CPU utilization
- Database throughput
- Overall cloud-server performance

Conclusion

For a cloud server experiencing high database-query latency, cache hierarchy and virtual memory serve different purposes. Cache hierarchy is primarily used to reduce memory-access latency, while virtual memory provides additional addressable memory through paging. Since paging to storage can introduce substantial delays, the recommended solution is to use an efficient L1–L2–L3 cache hierarchy, sufficient high-bandwidth DRAM, and fast storage, while minimizing unnecessary paging. This approach directly addresses the task's requirement to compare cache hierarchy and virtual-memory paging and select appropriate memory technologies based on latency and performance.

3. Autonomous Vehicle Data Processing

Problem Statement

An autonomous vehicle must process sensor data in real time. The memory system should therefore provide high bandwidth and low latency so that data from cameras, LiDAR, radar, and other sensors can be processed quickly. The task requires evaluating SRAM, DRAM, and MRAM and designing a memory hierarchy that maximizes bandwidth and minimizes latency.

1. Comparison of SRAM, DRAM and MRAM

Feature	SRAM	DRAM	MRAM
Full form	Static RAM	Dynamic RAM	Magnetoresistive RAM
Speed	Very high	High	High
Latency	Very low	Higher than SRAM	Low
Capacity	Small	Large	Medium
Power	Higher per bit	Requires refresh	Low standby power
Volatility	Volatile	Volatile	Non-volatile
Main use	CPU/cache memory	Main memory	Fast persistent/embedded memory
Suitability	Real-time critical data	Large sensor datasets	Frequently accessed/persistent data

2. Role of Each Memory

SRAM

SRAM is extremely fast and has very low latency. It is suitable for storing **time-critical sensor data and frequently accessed processing data**.

For example:

Camera/LiDAR data → SRAM → Processor

This allows the processor to access important data quickly.

DRAM

DRAM provides much greater capacity than SRAM and is therefore suitable for storing **large volumes of sensor data**.

Autonomous vehicles generate large amounts of data from multiple sensors, so DRAM can act as the main working memory.

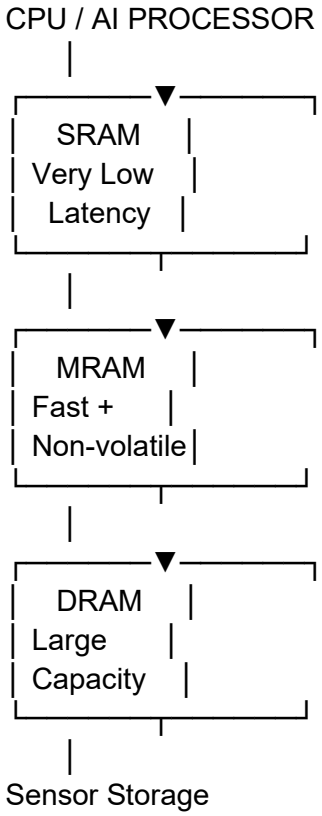
Sensors → DRAM → AI Processing

MRAM

MRAM is a **non-volatile memory** that provides fast access while retaining information without continuous power. It can be used for frequently required data, configuration information, and selected AI/vehicle-processing information where fast access and persistence are useful.

3. Proposed Memory Hierarchy

A suitable architecture is:



4. Working of the Proposed Design

The memory hierarchy can be organized according to the importance and frequency of data access.

Step 1 – Sensor Data Generation

Cameras, LiDAR, radar and other sensors continuously generate data.

Step 2 – SRAM

Critical and frequently accessed data is placed in SRAM so that the AI processor can access it with minimum latency.

Step 3 – MRAM

Important intermediate information, configuration data and frequently required information can be stored in MRAM because it provides fast access and retains data without power.

Step 4 – DRAM

Large amounts of sensor information and AI-processing data are stored in DRAM because it provides much greater capacity.

5. How the Design Maximizes Bandwidth

The system can improve bandwidth by:

- Using SRAM for the most time-critical data.
- Using high-bandwidth DRAM for large sensor datasets.
- Keeping frequently accessed information closer to the processor.
- Reducing unnecessary movement of data between memory levels.
- Using parallel memory access where possible.

This allows the processor to process multiple sensor streams efficiently.

6. How the Design Minimizes Latency

Latency can be reduced by keeping the most frequently accessed data in the fastest memory.

Most frequently accessed

↓

SRAM

↓

MRAM

↓

DRAM

↓

Less frequently accessed

7. Example in an Autonomous Vehicle

Suppose an autonomous vehicle suddenly detects an obstacle.

Camera + LiDAR

↓

Sensor Data

↓

SRAM

↓

AI Processor

↓

Obstacle detected

↓

Brake / Steering decision

Because critical sensor information is available in very fast memory, the processing delay can be minimized.

8. Expected Benefits

The proposed hierarchy provides:

Low latency

- SRAM provides very fast access for critical data.
- MRAM provides fast access to selected persistent information.

High bandwidth

- DRAM provides large-capacity, high-throughput working memory.
- Data movement between memory levels can be reduced.

Better real-time processing

- Critical sensor data is kept close to the processor.
- The AI system can process sensor information faster.

Reliability

- MRAM's non-volatile nature allows selected information to remain available even without continuous power.

Conclusion

For an autonomous vehicle, a combination of SRAM, MRAM and DRAM provides a suitable hierarchical memory architecture. SRAM should be placed closest to the processor for critical, frequently accessed data, MRAM can handle fast and persistent information, and DRAM should provide large-capacity storage for sensor and AI-processing data. This hierarchy helps maximize bandwidth and minimize latency, which is essential for real-time autonomous-vehicle decision making. The assessment specifically requires evaluation of these three technologies and a hierarchy designed around bandwidth and latency.

4. AI Inference Edge Device

Problem Statement

An edge AI device needs fast model loading with low power consumption. The task is to compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

1. Comparison of NAND Flash, DRAM and RRAM

Feature	NAND Flash	DRAM	RRAM
Type	Non-volatile	Volatile	Non-volatile
Speed	Slower	Fast	Fast
Latency	High	Low	Very low/low
Capacity	High	Medium/High	Medium
Power consumption	Low when idle	Requires refresh	Low
Main use	Permanent model storage	Working memory	Fast AI model/data storage
Data retention	Yes	No	Yes

2. Role of Each Memory

NAND Flash:

Used for **permanent storage of AI models**, operating systems and large datasets. It retains data even when power is removed.

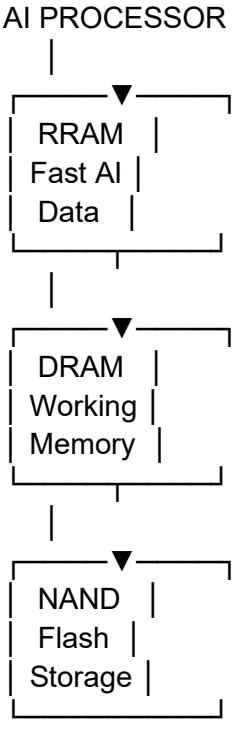
DRAM:

Used as the **main working memory** while the AI model is executing. It provides fast access to model parameters and intermediate computations.

RRAM:

RRAM can provide **fast, non-volatile storage** and can be useful for frequently accessed AI model parameters. Its non-volatile nature can reduce the need to repeatedly reload information from slower storage.

3. Proposed Hierarchical Memory Structure



4. Working

Step 1 – NAND Flash

The complete AI model is permanently stored in NAND Flash.

Step 2 – RRAM

Frequently required model parameters or important AI data can be placed in RRAM for faster access.

Step 3 – DRAM

During inference, the required model parameters and intermediate results are loaded into DRAM.

Step 4 – AI Processor

The processor accesses the required data from the faster memory levels to perform inference.

5. Power Optimization

The proposed hierarchy reduces power consumption by:

- Keeping permanent models in **NAND Flash** instead of continuously using DRAM.
- Using **RRAM** for frequently accessed non-volatile data.
- Loading only required model portions into DRAM.
- Reducing unnecessary data transfers between storage and working memory.
- Avoiding repeated model loading from slower NAND Flash.

6. Expected Benefits

The proposed structure provides:

Faster model loading:

Frequently used model information can be accessed from RRAM and DRAM.

Lower power consumption:

Non-volatile NAND Flash and RRAM retain data without continuous power.

Better inference performance:

DRAM provides fast working memory for the AI processor.

Efficient memory usage:

Large models can remain in NAND Flash while only the required portions are loaded into faster memory.

Conclusion

For an edge AI device, NAND Flash, RRAM and DRAM should be used together rather than relying on a single memory technology. NAND Flash provides high-capacity permanent storage, RRAM provides fast non-volatile access to important AI data, and DRAM provides fast working memory during inference. Therefore, the recommended hierarchy is:

AI Processor → RRAM → DRAM → NAND Flash

This design provides a balance between fast model access, low power consumption, storage capacity and inference performance, directly addressing the requirements of the given industry problem.

5. High-Performance Gaming Console

Problem Statement

A gaming console suffers frame drops while loading large game scenes. The problem requires analyzing the role of L3 cache and DRAM throughput and proposing memory-hierarchy enhancements to improve scene-loading performance.

1. Role of L3 Cache and DRAM

Feature	L3 Cache	DRAM
Type	Cache memory	Main memory
Speed	Very high	Lower than cache
Capacity	Smaller	Much larger
Latency	Low	Higher
Main purpose	Keep frequently used data close to CPU	Store large game data and active scenes
Effect on gaming	Reduces CPU waiting time	Provides large working space

2. Why Frame Drops Occur

Large game scenes contain:

- Textures
- 3D models
- Audio
- Animation data
- Lighting information
- Game objects

When a new scene is loaded, a large amount of data may need to be transferred from DRAM to the processor.

If the required data is not available in cache:

CPU
↓
L1/L2 Cache
↓
L3 Cache Miss
↓
DRAM
↓
Large data transfer
↓
Processing delay
↓
Frame Drop

Therefore, insufficient memory bandwidth or excessive DRAM accesses can affect smooth gameplay.

3. L3 Cache vs DRAM

L3 Cache

L3 cache is much closer to the processor and provides faster access. Frequently used game data and instructions can benefit from remaining in the cache.

However, L3 cache has **limited capacity**, so it cannot store an entire large game scene.

DRAM

DRAM provides much greater capacity and is suitable for storing large game scenes. However, it has higher access latency than cache.

Therefore:

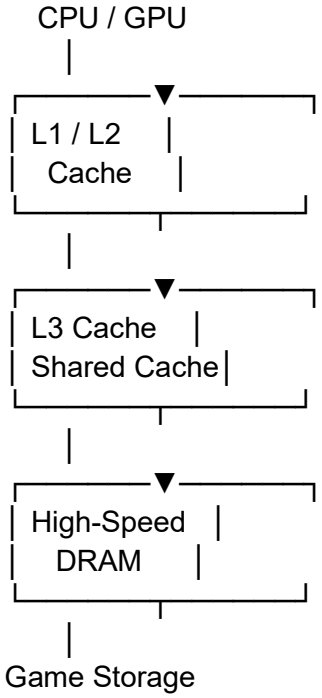
L3 = Speed

DRAM = Capacity + Bandwidth

Both are required for high-performance gaming.

4. Proposed Memory Hierarchy

A suitable hierarchy is:



5. Memory-Hierarchy Enhancements

a) Optimize L3 Cache

Increase the effectiveness of L3 cache so frequently reused game data remains available closer to the processor.

b) Increase DRAM Bandwidth

Use high-bandwidth DRAM so large textures, models and other scene data can be transferred quickly.

c) Improve Data Prefetching

The system can predict which game data will be required next and load it into cache or DRAM before it is needed.

d) Reduce Unnecessary Data Transfers

Frequently reused data should remain in faster memory rather than repeatedly being fetched from lower memory levels.

e) Use Efficient Memory Allocation

Game data should be organized so that frequently accessed data is placed in appropriate memory regions, reducing memory-access delays.

6. Example

Suppose the player enters a new large game environment.

Without optimization:

New Scene
↓
Large DRAM Data Transfer
↓
CPU/GPU waits
↓
Frame rate decreases
↓
Frame Drop

With the proposed hierarchy:
New Scene
↓
Frequently used data → L3 Cache
↓
Large scene data → High-speed DRAM
↓
Fast transfer to CPU/GPU
↓
Smooth Rendering

7. Expected Improvements
The proposed system can provide:

- Lower memory-access latency
- Higher DRAM throughput
- Faster scene loading
- Reduced CPU/GPU waiting
- Fewer frame drops
- Smoother gameplay

Conclusion
L3 cache and DRAM perform different but complementary functions in a gaming console. **L3 cache provides fast access to frequently used data**, while **DRAM provides the capacity and bandwidth required for large game scenes**. By optimizing L3 cache usage, increasing DRAM bandwidth, applying prefetching and reducing unnecessary data movement, the console can load large scenes more efficiently and reduce frame drops. This directly addresses the given industry problem of improving gaming performance through memory-hierarchy enhancements.